САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет программной инженерии и компьютерной техники (ПИиКТ)
Дисциплина: «Расширенный курс по бэкенд-разработке в среде Node.JS»

ОТЧЕТ ПО ДОМАШНЕЙ РАБОТЕ №3
«Тестирование API средствами Postman»

Выполнил:
студент гр. К3340 (поток БР1.1)
Галилей Кирилл Дмитриевич

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

1. ВАРИАНТ

Вариант 1. «Приложение для бронирования столиков в ресторанах»

Ключевая функциональность системы согласно заданию:
- Вход (аутентификация с выпуском JWT-токенов доступа)
- Регистрация (создание аккаунтов клиентов и менеджеров с хэшированием паролей)
- Личный кабинет пользователя (профиль, контактные данные, история)
- Поиск ресторанов с фильтрацией по кухне, расположению и цене
- Страница с информацией о ресторане (меню, фото, отзывы, столики)
- История бронирований пользователя (создание, просмотр, отмена)

2. ЗАДАЧА

- реализовать тестирование API средствами Postman;
- написать тесты внутри Postman.

В результате хочу получить один комплексный сценарий по основному процессу в вашем приложении.

3. ХОД РАБОТЫ

3.1. Структура коллекции и переменные окружения

Разработаны файлы:
- RestaurantBooking.postman_collection.json (16 запросов)
- RestaurantBooking.postman_environment.json (переменные baseUrl, tokens, IDs)

3.2. Пошаговый сквозной сценарий тестирования

1. Register Manager (POST /api/v1/auth/register) -> 201 Created
2. Login Manager (POST /api/v1/auth/login) -> 200 OK, сохранение manager_token
3. Register Client (POST /api/v1/auth/register) -> 201 Created
4. Login Client (POST /api/v1/auth/login) -> 200 OK, сохранение client_token
5. Create Restaurant (POST /api/v1/restaurants) -> 201 Created, сохранение restaurant_id
6. Add Table (POST /api/v1/restaurants/:id/tables) -> 201 Created, сохранение table_id
7. Add Menu Item (POST /api/v1/restaurants/:id/menu) -> 201 Created
8. Search Restaurants (GET /api/v1/restaurants?cuisine=Italian) -> 200 OK
9. Get Restaurant Details (GET /api/v1/restaurants/:id) -> 200 OK
10. Get Restaurant Menu (GET /api/v1/restaurants/:id/menu) -> 200 OK
11. Create Reservation (POST /api/v1/reservations) -> 201 Created, сохранение reservation_id
12. Overbooking Check (POST /api/v1/reservations) -> 409 Conflict (негативный тест)
13. Get My Reservations (GET /api/v1/reservations/my) -> 200 OK
14. Leave Review (POST /api/v1/restaurants/:id/reviews) -> 201 Created
15. Cancel Reservation (PATCH /api/v1/reservations/:id/cancel) -> 200 OK, статус CANCELLED
16. Unauthorized Check (GET /api/v1/users/profile) -> 401 Unauthorized (негативный тест)

3.3. Тестовые проверки

Каждый запрос содержит тесты pm.test:
- проверка HTTP статус-кодов (200, 201, 401, 409)
- проверка структуры JSON и типов возвращаемых полей

- валидация сохранения переменных окружения

3.4. Результаты запуска
В Collection Runner выполнено 16 запросов и 42 теста, 100% тестов успешно пройдены (0 failed).

4. ВЫВОД

В ходе выполнения домашней работы №3 разработан комплексный автоматизированный сценарий интеграционного тестирования API в Postman. Коллекция покрывает полный жизненный цикл бронирования столиков, включая позитивные цепочки действий двух независимых акторов (менеджера и клиента) и негативные проверки (овербукинг и отсутствие авторизации). Сценарий полностью автономен и выполняется в Collection Runner без ручного вмешательства.